

课程笔记

Codex 中的 Plan Mode 与 update_plan

基于五道口纳什 Modern Agent 系列整理

2026

Codex Plan Mode update_plan

视频作者/频道：五道口纳什

发布日期：2026-04-09

视频时长：14:49

视频链接：<https://www.bilibili.com/video/BV1NdDtBjEg7/>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言：为什么 Planning 是 Coding Agent 的核心能力	2
1.1 本章小结	2
2 Plan Mode：通过协作生成规格	3
2.1 Plan Mode 的适用场景	3
2.2 本章小结	4
3 update_plan：执行期的 Live Progress Model	4
3.1 本章小结	5
4 为什么这种设计很优雅	5
4.1 与文件化计划的关系	6
4.2 本章小结	6
5 实践建议：什么时候用哪一种计划	7
5.1 本章小结	7
6 总结与延伸	7

1 引言：为什么 Planning 是 Coding Agent 的核心能力

本讲继续讨论 Codex / Claude Code 这一类 coding agent 的工具设计。讲者指出，前面介绍 Codex 的最小工具集时，容易漏掉一个看似简单但非常关键的工具：`update_plan`。在实际开发任务里，一个 agent 面对的不是单步问答，而是一个不断展开的代码库任务：先理解 codebase，再形成计划，然后执行、检查、调整，直到任务完成。

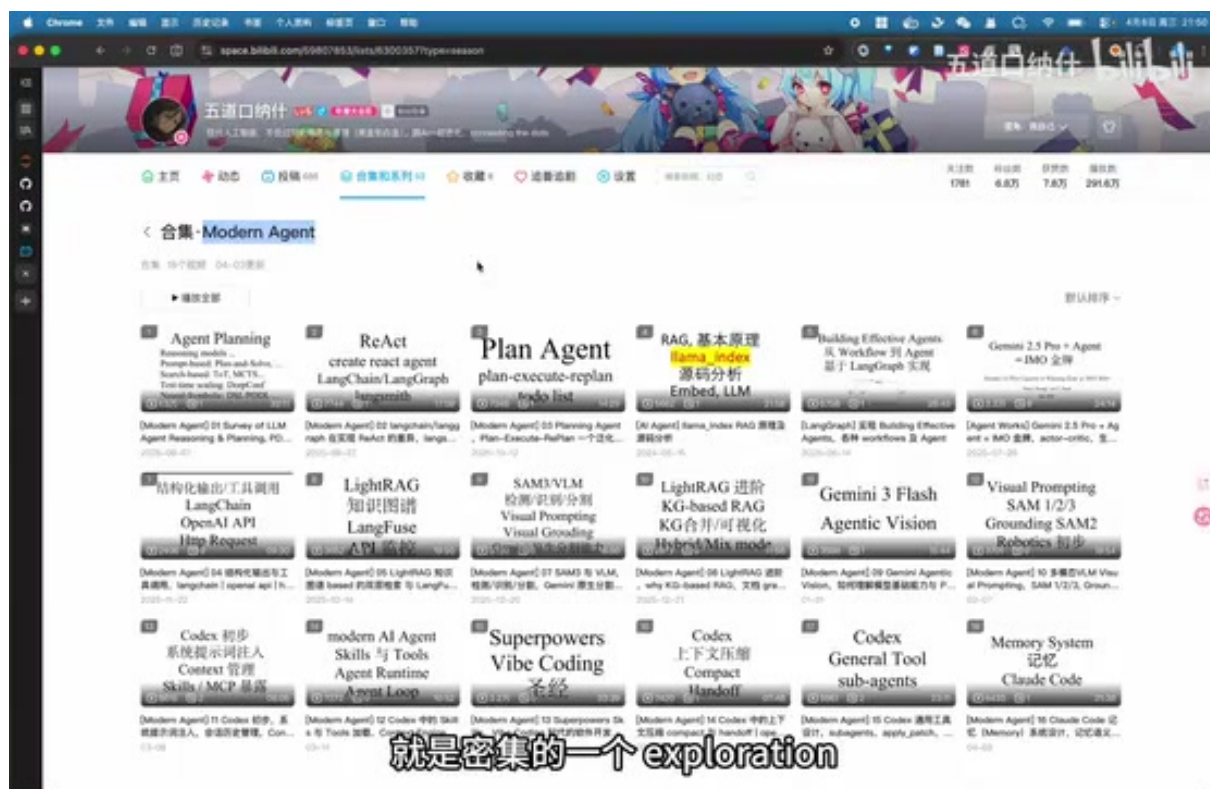


图 1: Modern Agent 系列中，Planning Agent 与 Codex 工具设计形成一条连续线索。¹

Codex 的典型任务流

在讲者的抽象里，Codex 处理一个开发需求通常经历三步：**codebase exploration**、**plan**、**execution**。其中 exploration 负责密集读取代码与上下文，plan 把目标拆成可执行步骤，execution 再根据执行反馈持续推进。

这也是为什么 planning 不只是一个 UI 功能。它实际上是 agent loop 中的状态表示：模型需要把“当前要做什么”、“已经做到哪一步”、“是否需要重新规划”保留在上下文里，并在必要时展示给用户。

1.1 本章小结

本讲讨论的是 Codex 里两套互补的规划机制：一种是显式的 Plan Mode，另一种是执行期的 `update_plan` 工具。前者偏协作和需求澄清，后者偏运行时进度管理。

¹ 视频画面时间区间：00:00:45–00:00:55。

2 Plan Mode: 通过协作生成规格

Codex 的第一种规划方式是 Plan Mode。用户可以通过 `/plan` 进入这个模式。进入之后，系统会在 developer message 中植入一组完整的行为约束，让模型先探索、再澄清、再形成一份可执行的 Markdown 计划。

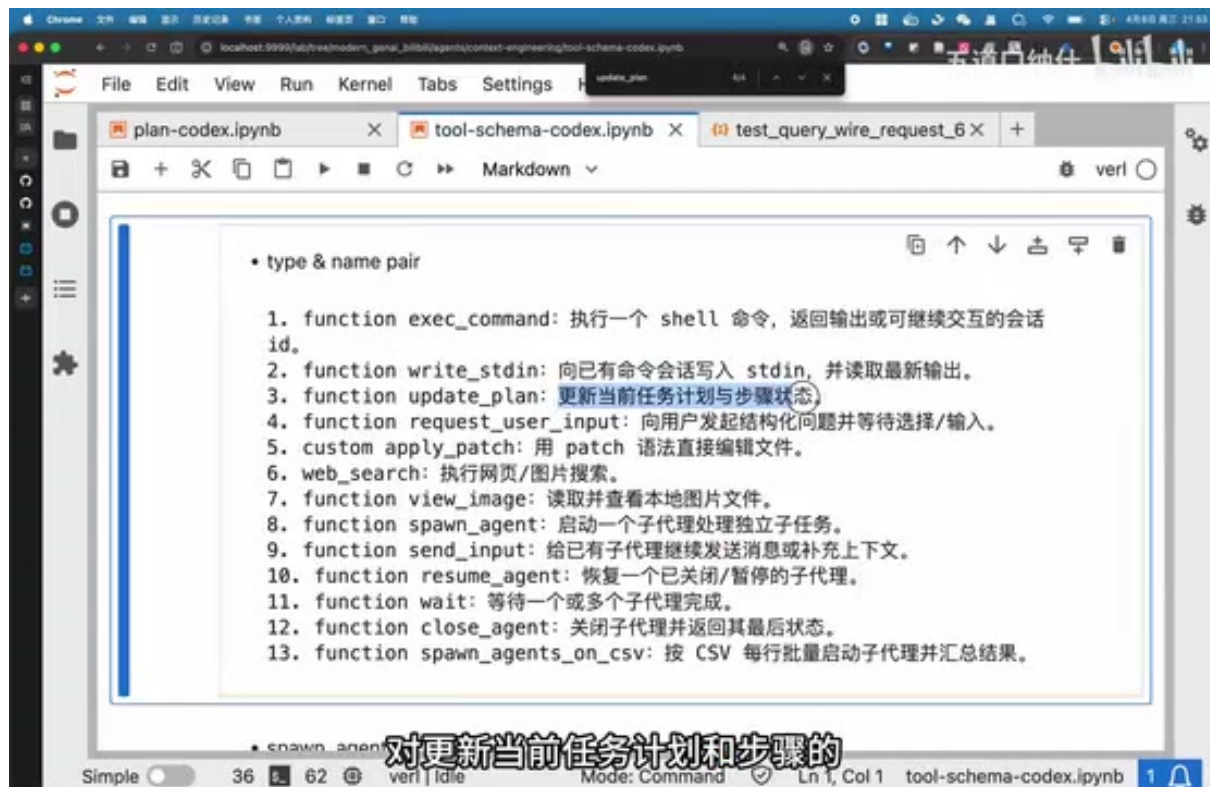


图 2: Plan Mode 的核心不是立刻执行，而是通过对话形成可实施规格。²

Plan Mode 与一个特殊工具配合：`request_user_input`。这个工具只在 Plan Mode 中可用，用来向用户提出一到三个短问题，等待用户回答，从而减少需求不清导致的错误实现。

Plan Mode 的产物在哪里？

讲者特别强调：Plan Mode 结束后形成的是一段 Markdown 文本，但它并不会自动落盘成本地文件。它存在于 context window 中，作为后续执行阶段的上下文。换句话说，这是一份上下文内的规格，而不是仓库里的持久化文档。

Plan Mode 的约束也很强：在这个阶段应该做非变更式探索，不直接修改代码。它更像是一次需求澄清和规格设计会话，最后再询问用户是否执行这份计划。用户确认后，系统回到默认执行模式。

2.1 Plan Mode 的适用场景

Plan Mode 适合需求边界还不清楚、实现路径有多个分叉、或者需要用户选择取舍的任务。例如一个大型重构可能涉及 API 兼容性、数据迁移和测试策略，此时先让模型用问题收敛规格，比直

²视频画面时间区间：00:03:25-00:03:45。

接动手更稳。

Plan Mode 不是持久化项目管理工具

Plan Mode 的计划只是上下文对象。它可以指导当前会话，但如果希望跨会话保存、恢复或审计，仍然应该像本仓库一样使用文件化计划，例如 `task_plan.md`、`NOTE_GENERATION_TODO.md` 等。

2.2 本章小结

Plan Mode 是一种协作模式。它通过 `developer instruction` 和 `request_user_input` 约束模型行为，把“先问清楚再写代码”产品化成一个明确流程。

3 update_plan: 执行期的 Live Progress Model

第二种规划方式是 `update_plan`。它不是进入或退出 Plan Mode 的开关，而是一个执行期工具，用于维护当前任务的 checklist / todo list。讲者把它称为一种 live progress model，即 agent 对当前进度状态的内部表示，同时也可以显示给用户。

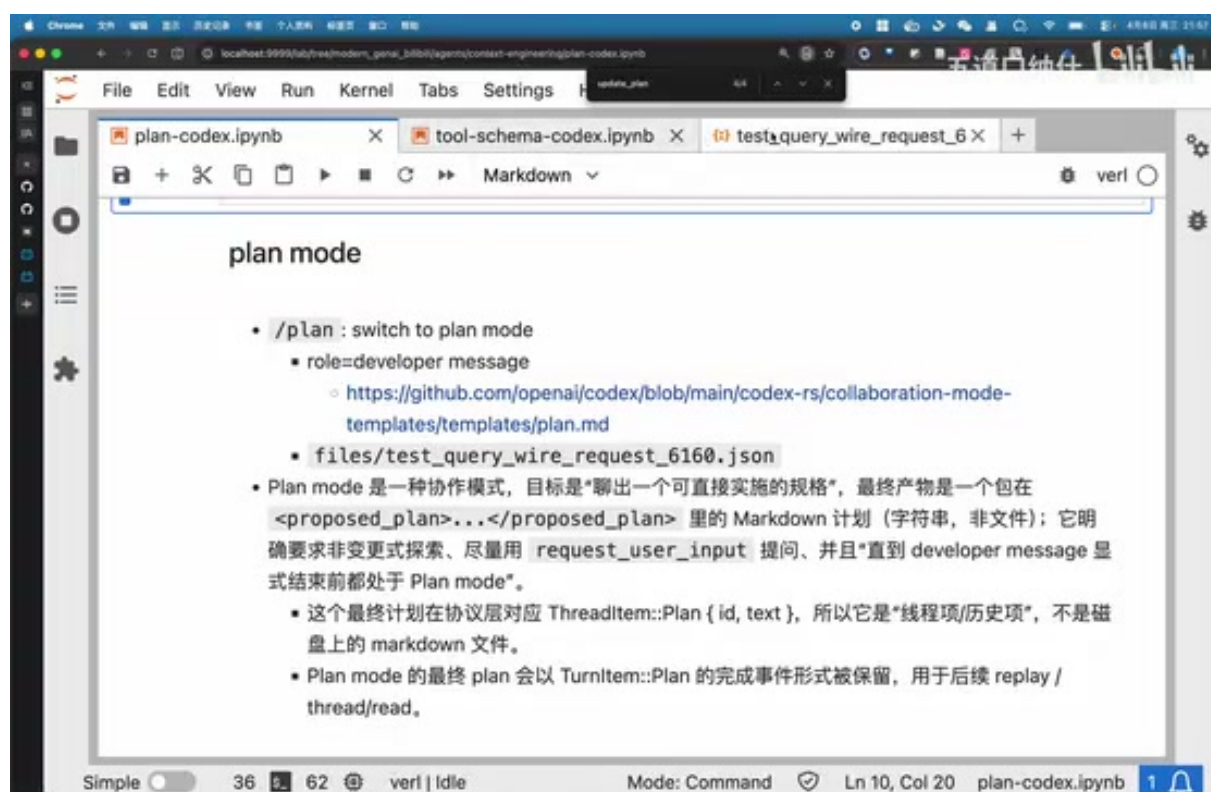


图 3: `update_plan` 的 schema 很小：解释字段加上带状态的步骤列表。³

`update_plan` 的 schema 非常简洁：

1 {

³视频画面时间区间：00:06:20–00:06:40。

```

2  "explanation": "Why the plan is being updated",
3  "plan": [
4    {"step": "Inspect source files", "status": "in_progress"},
5    {"step": "Patch implementation", "status": "pending"},
6    {"step": "Run verification", "status": "pending"}
7  ]
8  }

```

Listing 1: update_plan 的概念结构

其中每个 step 有三种状态: pending、in_progress、completed。系统约束最多只能有一个 step 处于 in_progress，这保证计划始终有清晰的当前焦点。

同一个工具承载三种动态

update_plan 同时承载 plan 的创建、状态更新和 replan。它没有单独的 createPlan、modifyStep 或 replan API，而是每次提交一个新的完整计划快照。

这就是讲者反复强调的“plan dynamics”：第一次调用相当于创建计划；后续调用可以把某一步从 pending 推进到 in_progress 或 completed；如果执行中发现原计划不可行，也可以重新提交一份新的计划快照，并在 explanation 中说明原因。

3.1 本章小结

update_plan 的关键不是复杂 API，而是极简状态模型。模型通过反复提交计划快照，在同一个 agent loop 中完成创建、更新和重规划。

4 为什么这种设计很优雅

讲者把 Codex 的这套设计称为一种 general、unified、minimal 的 tool system。原因在于，一个很小的函数接口就能表达复杂规划行为，而复杂度被交给模型的语义能力处理。

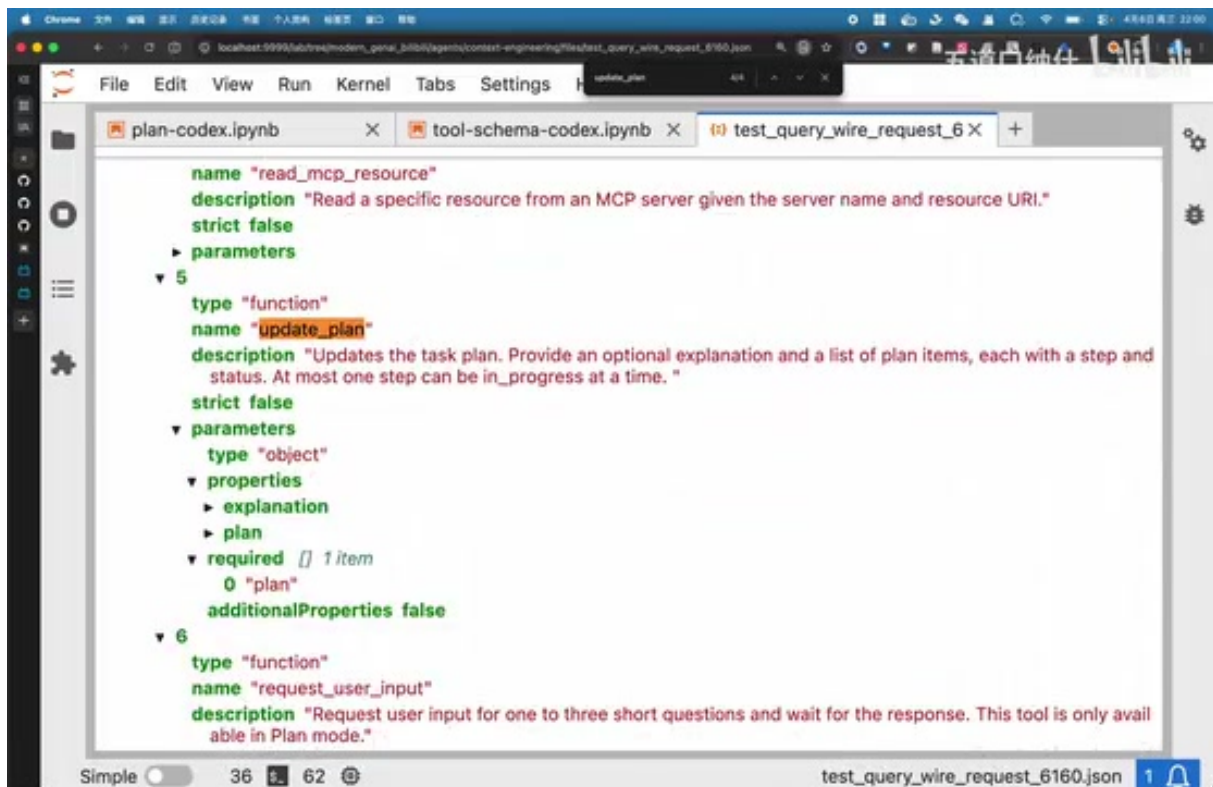


图 4: 讲者对比 Codex 的工具 schema, 强调其统一性和极简性。⁴

这种设计的好处有三点:

1. **接口小**: 工具层只暴露一个 `update_plan`, 不需要为每种计划操作新增 API。
2. **语义强**: 创建、更新、重规划都由模型通过参数语义区分。
3. **用户可见**: 计划不是隐藏在模型内部, 而是能作为进度反馈展示出来。

快照而不是补丁

`update_plan` 更像是提交“当前计划状态快照”, 而不是对某个 step 发送 patch。这让模型在 replan 时可以整体重排步骤, 也让 UI 或日志系统更容易渲染当前状态。

4.1 与文件化计划的关系

需要注意的是, `update_plan` 仍然是上下文级进度工具。对于长任务、跨会话任务、批量生成任务, 文件化计划仍然很有必要。上下文内 plan 适合即时执行, 文件化 todo 适合恢复、审计和团队协作。

4.2 本章小结

Codex 的设计把规划能力压缩到一个极小但表达力足够的工具里。随着 GPT-5.4 这类模型的能力增强, 模型可以非常顺滑地用这个工具维护计划状态。

⁴视频画面时间区间: 00:09:25-00:09:45。

5 实践建议：什么时候用哪一种计划

结合本讲内容，可以把两种机制分工如下：

机制	主要用途	典型场景
Plan Mode	澄清需求、形成规格	用户目标模糊、任务路径多、需要选择取舍
update_plan	执行中进度管理	已经开始做任务，需要展示状态和动态调整
文件化 todo	跨会话持久化	长任务、批量任务、需要恢复和审计

表 1: 三种计划载体的分工。

对于 coding agent 来说，最稳的工作流通常是：先探索代码库，必要时进入 Plan Mode 澄清规格；执行阶段用 update_plan 管理当前进度；如果任务跨度很大，则额外维护文件化计划。

计划不是越细越好

计划的价值在于减少不确定性，而不是制造仪式感。过细的计划会让 agent 在低价值步骤上浪费上下文；过粗的计划又无法指导执行。好的计划应该只捕捉真正影响执行顺序和风险控制步骤。

5.1 本章小结

Plan Mode、update_plan 和文件化 todo 是三个层次。它们分别解决协作澄清、执行状态和长期持久化问题。

6 总结与延伸

本讲的核心结论可以压缩成一句话：Codex 的 planning 能力不是单一按钮，而是一组分层机制。Plan Mode 负责在动手前把规格聊清楚；update_plan 负责在执行中维护 live progress；文件化 todo 则负责把长期状态从 context window 中解耦出来。

讲者最后指出，随着模型能力增强，像 update_plan 这样极简的工具已经足以让模型完成复杂的创建、状态推进和 replan。真正重要的不是工具数量，而是工具 schema 是否和模型的语义能力对齐。

最终 takeaway

优秀的 agent harness 不一定要堆很多专用工具。一个足够通用、语义清晰、状态可视的工具，配合强模型，往往能覆盖更大的行为空间。Codex 的 update_plan 就是这种设计的代表。

对于我们维护课程笔记仓库，也可以借鉴这套思路：短任务用上下文计划推进，长任务用文件化 todo 固化状态；每次遇到下载、字幕、转录、编译等阻塞，都不要只停在口头记忆里，而要写回队列，形成可恢复的工作流。